Neural Network Model Compression: A Benchmark Study On Pruning And Low-Rank Factorization

Dev Patel
Department of Computer Science
North Carolina State University
dmpatel3@ncsu.edu

Abstract

Model compression is a technique that involves decreasing the size and complexity of neural networks in order to create smaller versions of those networks. While there have been a variety of methods used for compressing a network, the compression techniques that are commonly used include pruning and low-rank factorization. This paper focuses on evaluating these two techniques independently of each other in terms of their effectiveness in maximizing the compression for a given network while minimizing the accuracy loss with the compressed network by implementing them on three different sets of two models for three different machine learning (ML) tasks.

1 Introduction

Neural networks are a common AI model used for completing various machine learning (ML) tasks such as machine translation and voice recognition [1]. However, in order for these networks to be able to get the performance needed to do these tasks accurately, they often need to be big and complex in order to be able to learn and retain the knowledge needed for these tasks. Because of this, these models typically require a lot of memory storage and computing power in order to run properly, which can make it difficult to apply such models on smaller devices like phones and laptops. To help with resolving this problem, model compression has become a common solution used for making such models compatible on devices with computing limitations. Model compression is a technique that involves decreasing the size and complexity of neural networks in order to create smaller variations of those networks. However, when a network is compressed, the smaller model is usually retrained to help the compressed model make up for some of the accuracy that was lost as a result of the compression [4]. While there have been a variety of methods used for compressing a network, the techniques that are commonly used include knowledge distillation, weight sharing, pruning, quantization, and low-rank factorization [4].

2 Motivation

Even though model compression allows existing networks to become usable with resource-constrained devices, making the models smaller does cause those models to lose some accuracy due to their smaller size and decreased complexity. As a result, this creates an optimization problem with maximizing how much a model can be compressed and minimizing the accuracy that's lost with the smaller model, although it's possible for the compressed models to achieve a higher accuracy than the original models if the compression is optimized well. Despite this problem that arises with using model compression, some techniques are likely able to do better than others in terms of optimizing this tradeoff. For this paper, the pruning and low-rank factorization methods will be evaluated independently of each other to generate a current benchmark on their performance relative

40th Conference on Neural Information Processing Systems (NeurIPS 2026).

to each other in terms of how well they maximize the compression for a network and minimize the accuracy that's lost with the compressed network.

3 Previous Work

There has been a lot of work recently done on applying compression methods to a variety of models such as quantization on transformer models [5] and pruning on large language models (LLMs) [9]. Additionally, there's been work done that focuses on combining different techniques together such as low-rank factorization and pruning [12] and knowledge distillation and quantization [8]. However, there hasn't been as much work that compares compression techniques directly with each other and evaluate their performance to one another. In addition, the pruning and quantization techniques in particular have been explored more in recent works whereas other methods like low-rank factorization and knowledge distillation haven't been explored as much recently. A paper of noteworthy significance is one that was published in 2023 [1] which compares the performance of quantization and pruning independently of each other on different models used for four different tasks like image classification and object detection. However, this paper only focused on comparing these techniques on CNNs and didn't compare their performance on other model types like RNNs and transformers. Furthermore, the paper didn't explore low-rank factorization as a method to be used for comparison and only compared pruning and quantization.

4 Model Compression Techniques

4.1 Pruning

Pruning is a technique that involves removing a certain number of weights at certain locations in a pretrained model to reduce the number of parameters in the model [4]. To do this, a threshold of a certain magnitude can be defined individually for each layer or defined globally for the whole model to remove weights with magnitudes that are lower than the threshold [4]. Alternatively, a percentage of weights to be removed can be defined for each layer instead of using a threshold, where the percentage of weights that are closest to zero are removed [4]. Additionally, there are also unstructured and structured versions of pruning, where unstructured focuses on freely removing individual weights to improve performance at the cost of extra computation time while structured focuses on removing specific groups of weights to improve computation time at the cost of degraded performance [4].

4.2 Low-Rank Factorization

Low-rank factorization is a technique that involves decomposing the matrices defining the weights into a lower rank approximation as a way of removing redundant parameters in the model [4]. The idea behind low-rank factorization is that a weight matrix A of size m x n and rank r can be decomposed as A = WH, where W and H are two separate matrices of sizes m x r and r x n, which helps reduce the amount of space needed to store the weight matrices [4]. With this in mind, low-rank factorization focuses on finding a low rank k < r that optimizes the function

$$\min_{\mathbf{W},\mathbf{H}} \frac{1}{2}||\mathbf{A} - \mathbf{WH}||_F^2$$

where $||\cdot||_F$ is the Frobenius norm, and W and H are defined with sizes m x k and k x n, and singular value decomposition (SVD) is usually used to find the low rank, although other decomposition methods such as Canonical Polyadic decomposition (CPD) and Tucker decomposition have also been used [4].

5 Experiments

5.1 Setup

For this paper, the networks that were chosen for testing can be categorized as a convolutional neural network (CNN) or recurrent neural network (RNN). Additionally, three different ML tasks (Audio

Classification, Sentiment Analysis, and Machine Translation) were explored using a set of two models unique to each of them (A CNN and RNN each) along with using a dataset tailored to each task. The models and dataset chosen for the three tasks were assigned as the following (Models assigned to each task are listed as the categories [CNN, RNN] in that order):

- Audio Classification - M3 and AudioLSTM with the Google Speech Commands dataset
- Sentiment Analysis - TextCNN and BiLSTM with the IMDB dataset
- Machine Translation - ConvS2S and GRUSeq2Seq with the IWSLT2017 dataset

Papers like [7], [11], [3], and [2] provide details on the M3, TextCNN, ConvS2S and GRUSeq2Seq model respectively along with GitHub repos containing the implementation for these models, so these were used to setup and train these four models. For the AudioLSTM and BiLSTM models, there were existing GitHub [10] and Kaggle [6] repos that includes the implementation of these two models respectively, so these were used to setup and train the remaining two models. For evaluating pruning and low-rank factorization, unstructured percentage-based pruning and factorization using SVD were chosen to be applied on the models as these implementations of the two techniques have been commonly used in past model compression works. In addition, a common set of compression rates (15, 30, 45, 60, 75) were applied to each technique, and the NVIDIA GeForce RTX 2080 Ti GPU that's accessible through the North Carolina State University Virtual Computing Lab (NCSU VCL) was used for training and testing the base and compressed models, with PyTorch being used to implement the model training and testing. In terms of model training, the original models were trained with five epochs, and the compressed models were retrained for only one epoch. For training and testing the original versions of the six models, the training parameter values typically used for those models were applied, and those same parameter values were used for training the compressed versions of those models.

5.2 Results

All models were evaluated on model size (defined by the total number of non-zero parameters present in the model) and test accuracy gain/loss.

Table 1: Pruning and low-rank factorization results for the 6 models tested. Bolded rows indicate the compression rate where the model achieved the best accuracy gain for the technique used.

M3 (Pruning)		
Compression Rate	Model Size (Number Of Non-Zero Parameters)	Accuracy Gain Or Loss
0 (Original)	67907	-
0.15	57884	-0.0341
0.30	47862	0.1752
0.45	37840	0.1261
0.60	**27818**	**0.2130**
0.75	17795	0.2055
M3 (Factorization)		
Compression Rate	Model Size (Number Of Non-Zero Parameters)	Accuracy Gain Or Loss
0 (Original)	67907	-
0.15	94218	0.1021
0.30	**77611**	**0.2458**
0.45	61164	0.1972
0.60	44557	0.2145
0.75	28379	0.1649
AudioLSTM (Pruning)		
Compression Rate	Model Size (Number Of Non-Zero Parameters)	Accuracy Gain Or Loss
0 (Original)	235939	-

0.15	**235267**	**0.0262**
0.30	234595	0.0230
0.45	233923	0.0172
0.60	233251	0.0131
0.75	232579	0.0073

AudioLSTM (Factorization)

Compression Rate	Model Size (Number Of Non-Zero Parameters)	Accuracy Gain Or Loss
0 (Original)	235939	-
0.15	236186	0.0223
0.30	235371	0.0207
0.45	**234556**	**0.0227**
0.60	233741	0.0058
0.75	232763	0.0075

TextCNN (Pruning)

Compression Rate	Model Size (Number Of Non-Zero Parameters)	Accuracy Gain Or Loss
0 (Original)	3354374	-
0.15	3331244	0.0138
0.30	3308114	0.0017
0.45	**3284984**	**0.0140**
0.60	3261854	0.0132
0.75	3238724	0.0104

TextCNN (Factorization)

Compression Rate	Model Size (Number Of Non-Zero Parameters)	Accuracy Gain Or Loss
0 (Original)	3354374	-
0.15	3356834	0.0047
0.30	**3329264**	**0.0113**
0.45	3301754	0.0111
0.60	3274214	0.0044
0.75	3246674	0.0064

BiLSTM (Pruning)

Compression Rate	Model Size (Number Of Non-Zero Parameters)	Accuracy Gain Or Loss
0 (Original)	5568386	-
0.15	5568232	0.0054
0.30	5568079	0.0058
0.45	**5567925**	**0.0076**
0.60	5567772	0.0020
0.75	5567618	0.0038

BiLSTM (Factorization)

Compression Rate	Model Size (Number Of Non-Zero Parameters)	Accuracy Gain Or Loss
0 (Original)	5568386	-
0.15	5568386	0.0070
0.30	5568386	0.0074
0.45	5568386	0.0085
0.60	**5568386**	**0.0090**
0.75	5568386	-0.0039

ConvS2S (Pruning)

Compression Rate	Model Size (Number Of Non-Zero Parameters)	Accuracy Gain Or Loss
0 (Original)	20582720	-
0.15	**18152148**	**0.0143**
0.30	15721584	0.0003
0.45	13291024	0.0076

0.60	10860460	0.0048
0.75	8429888	-0.0244

ConvS2S (Factorization)		
Compression Rate	Model Size (Number Of Non-Zero Parameters)	Accuracy Gain Or Loss
0 (Original)	20582720	-
0.15	25987968	0
0.30	22170112	0.0110
0.45	**18355264**	**0.0278**
0.60	14537408	0.0058
0.75	10740032	0.0262

GRUSeq2Seq (Pruning)		
Compression Rate	Model Size (Number Of Non-Zero Parameters)	Accuracy Gain Or Loss
0 (Original)	14201664	-
0.15	12923674	0.0109
0.30	11645682	0.0132
0.45	**10367694**	**0.0137**
0.60	9089702	0.0092
0.75	7811712	0.0006

GRUSeq2Seq (Factorization)		
Compression Rate	Model Size (Number Of Non-Zero Parameters)	Accuracy Gain Or Loss
0 (Original)	14201664	-
0.15	13977280	0.0060
0.30	12509760	0.0140
0.45	11049216	0.0125
0.60	**9581696**	**0.0235**
0.75	8123200	0.0054

Model size was measured based on the number of non-zero parameters because it's a common metric that has been typically used for model compression experiments when evaluating a model's size. Additionally, since model compression techniques can either increase or decrease the performance of a given model, accuracy gains were noted as positive values while accuracy losses were noted as negative values.

Table 2: Performance results for pruning and low-rank factorization on the 6 tested models. Bolded rows indicate the models where the technique performed the best for the given model.

Pruning Performance			
Model	Compression Ratio	Best Accuracy Gain	Performance Ratio
M3	**0.4096**	**0.2130**	**0.5200**
AudioLSTM	**0.9972**	**0.0262**	**0.0263**
TextCNN	**0.9793**	**0.0140**	**0.0143**
BiLSTM	0.9999	0.0076	0.0076
ConvS2S	0.8819	0.0143	0.0162
GRUSeq2Seq	0.7300	0.0137	0.0188
Average	0.8330	0.0481	0.0578

Factorization Performance			
Model	Compression Ratio	Best Accuracy Gain	Performance Ratio
M3	1.1429	0.2458	0.2151
AudioLSTM	0.9941	0.0227	0.0228
TextCNN	0.9925	0.0113	0.0114

BiLSTM	**1**	**0.0090**	**0.0090**
ConvS2S	**0.8918**	**0.0278**	**0.0312**
GRUSeq2Seq	**0.6747**	**0.0235**	**0.0348**
Average	0.9493	0.0567	0.0597

Using the recorded accuracy gains/losses for each technique, the compression rate with the highest accuracy gain was extracted from each model, and the number of non-zero parameters at that rate was divided by the original number of non-zero parameters for the model to get a compression ratio (CR) for the model at that rate. Using the computed CR along with the accuracy gain recorded at that rate, the accuracy gain was divided by the CR in order to compute the performance ratio (PR) for the model using the given technique. Finally, the PR values calculated for all six models for the two techniques were averaged out in order to obtain the average performance ratio for each technique.

5.3 Analysis

Even though compressing a model typically causes some accuracy to be lost with the smaller model, the experiments showed the opposite occurring where the smaller models tended to gain some accuracy even at higher compression rates like 0.60 and 0.75. However, this may have been partly due to the training of the original models not being optimized for the GPU properly since the original models were only trained with five epochs, which likely lowered the best accuracies that could be achieved with the baseline models. Looking at the pruning results in Table 1, while there's a consistency of the model size decreasing as the rate increases, the difference in the number of non-zero parameters as the rate increased significantly differed between models as models such as BiLSTM and TextCNN didn't see as much reduction in the number of non-zero parameters across different rates while other models like M3 and ConvS2S saw significant reductions in the number of non-zero parameters across different rates. This same phenomenon was also seen with the factorization results, and factorization overall seemed to compress the model less than pruning as the model size tended to be higher for factorization compared to pruning when using the same compression rates on the same models. Furthermore, factorization seems to also potentially increase the size of the model at times as when the compression rate was at 0.15 or 0.3 for models such as M3 and ConvS2S, the model size was bigger than the original size as indicated with the higher number of non-zero parameters compared to the number of non-zero parameters with the original models. This could have possibly occurred if the computed rank r used to compute the two smaller matrices of sizes m x r and r x n during SVD was too large as this can make the total size of the two matrices greater than the size of the original matrix m x n. Because of this, what likely happened was that using low compression rates lead to the calculated rank being very high, which therefore made the combined size of the two smaller matrices larger than the original matrix size and the computed model size bigger as a result.

Examining the table 2 results shows that the overall accuracy gain tended to be pretty low for both techniques across the six models, only being around 1-2% except for the M3 model which saw an accuracy gain of roughly 20-25%. In a similar manner, the calculated compression ratios with both techniques across the six models tended to be high rather than low, with most of them being around 1 besides some noteworthy exceptions like seen with the GRUSeq2Seq and ConvS2S models. Additionally, the computed performance ratios across the six models with both techniques were fairly close to each other, only being different by about 0.02 on average, with the main exception being with the M3 model as pruning saw a performance ratio of 0.52 while factorization got a performance ratio of 0.22. Despite this, pruning had a lower average compression ration while factorization achieved a higher average accuracy gain, suggesting a tradeoff between how much a model can be compressed and the amount of accuracy that's gained for the compressed model between the two techniques. Ultimately, while the average performance ratios for both techniques were approximately equal and both methods performed the best on half of the models tested, low-rank factorization did obtain a slightly higher performance ratio, implying that low-rank factorization performs the best in terms of maximizing the compression for a network and accuracy gained with the compressed network.

Conclusion

While the performances of pruning and low-rank factorization were very similar, low-rank factorization does perform slightly better in terms of minimizing the size of a model while maximizing

the accuracy that's earned with the smaller model. However, although low-rank factorization overall seems to do better with allowing the compressed model to earn more accuracy, pruning seems to be more effective for decreasing the size of the model, implying a potential tradeoff between these two factors. Because of this, future research work could explore this tradeoff further with these two techniques by evaluating other common types of pruning and low-rank factorization used and seeing if this tradeoff can also be seen with these other types. In addition, other future work that could be done includes comparing other common model compression techniques such as quantization and knowledge distillation with the two techniques explored in this paper. Finally, future work could also focus on comparing the performance of different versions of pruning and low-rank factorization like structured pruning and factorization using Tucker decomposition and likewise for other compression techniques explored along with exploring the use of these techniques on other common neural network architectures like transformers.

References

[1] Andrey Kuzmin, Markus Nagel, Mart van Baalen, Arash Behboodi, and Tijmen Blankevoort. Pruning vs Quantization: Which is Better? *Advances In Neural Information Processing Systems*, 36, 2023.

[2] Biao Zhang, Deyi Xiong, and Jinsong Su. A GRU-Gated Attention Model for Neural Machine Translation. *arXiv preprint arXiv:1704.08430v2*, 2019.

[3] Hirokazu Kameoka, Kou Tanaka, Damian Kwasny, Takuhiro Kaneko, and Nobukatsu Hojo. ConvS2S-VC: Fully convolutional sequence-to-sequence voice conversion. *arXiv preprint arXiv:1811.01609v3*, 2020.

[4] James T. O'Neill. An Survey of Neural Network Compression. *arXiv preprint arXiv:2006.03669v2*, 2020.

[5] Junyi Wu, Haoxuan Wang, Yuzhang Shang, Mubarak Shah, and Yan Yan. PTQ4DiT: Post-training Quantization for Diffusion Transformers. *Advances In Neural Information Processing Systems*, 37, 2024.

[6] Suruthi S. Sentiment Analysis using Bi-LSTM. https://www.kaggle.com/code/suruthi41/sentiment-analysis-using-bi-lstm, 2023.

[7] Wei Dai, Chia Dai, Shuhui Qu, Juncheng Li, and Samarjit Das. Very Deep Convolutional Neural Networks for Raw Waveforms. *arXiv preprint arXiv:1610.00087v1*, 2016.

[8] Xinrui Chen, Yizhi Wang, Renao Yan, Tian Guan, and Yonghong He. TexQ: Zero-shot Network Quantization with Texture Feature Distribution Calibration. *Advances In Neural Information Processing Systems*, 36, 2023.

[9] Xinyin Ma, Gongfan Fang, and Xinchao Wang. LLM-Pruner: On the Structural Pruning of Large Language Models. *Advances In Neural Information Processing Systems*, 36, 2023.

[10] Yi Hung Wong. Speech-Commands-Classification-LSTM. https://github.com/yihong1120/Speech-Commands-Classification-LSTM, 2023.

[11] Yoon Kim. Convolutional Neural Networks for Sentence Classification. *arXiv preprint arXiv:1408.5882v2*, 2014.

[12] Yuchuan Tian, Hanting Chen, Tianyu Guo, Chao Xu, and Yunhe Wang. Towards Higher Ranks via Adversarial Weight Pruning. *Advances In Neural Information Processing Systems*, 36, 2023.